



Haute Ecole de la Province de Liège

Techniques avancées de gestion des données

Bases de données avancées et XML

Leçon 4 : SAX et DOM

Samuel Hiard

Année académique 2023-2024

Agenda

- Rappel : validation
- SAX
 - Principe
 - Validation avec SAX
- DOM
 - Principe
 - Navigation et manipulation du DOM

Agenda

- Rappel : validation
- SAX
 - Principe
 - Validation avec SAX
- DOM
 - Principe
 - Navigation et manipulation du DOM

Rappel : Validation

- Un document XML vous laisse faire « ce que vous voulez ».
- Pas de contrainte sur la structure
- Comment vérifier si le document est bien écrit?
- Règles implicites de XML
 - Une balise ouvrante doit toujours correspondre avec une balise fermante
 - Règles de syntaxe
 - → Bien rédigé (well-formed)
- Schéma de validation
 - Un peu comme un schéma relationnel
 - Précise les règles de structure (tel élément est composé de tels autres éléments, attributs, etc.)
 - DTD ou XSD
 - → Valide (valid)

Rappel : Validation (2)

- Comment vérifier la validité d'un fichier XML avec un DTD ou un XSD?
- Avec un *parser*
- Deux approches : SAX et DOM
- SAX (Simple API for XML)
 - Parcourt le fichier XML et génère des événements (début de balise, fin de balise, etc)
 - Moins gourmand en ressources
 - Lecture séquentielle
- DOM (Document Object Model)
 - Génération d'un arbre en mémoire et parcourt de ce dernier
 - Plus gourmand en ressources
 - Parcourt de l'arbre possible

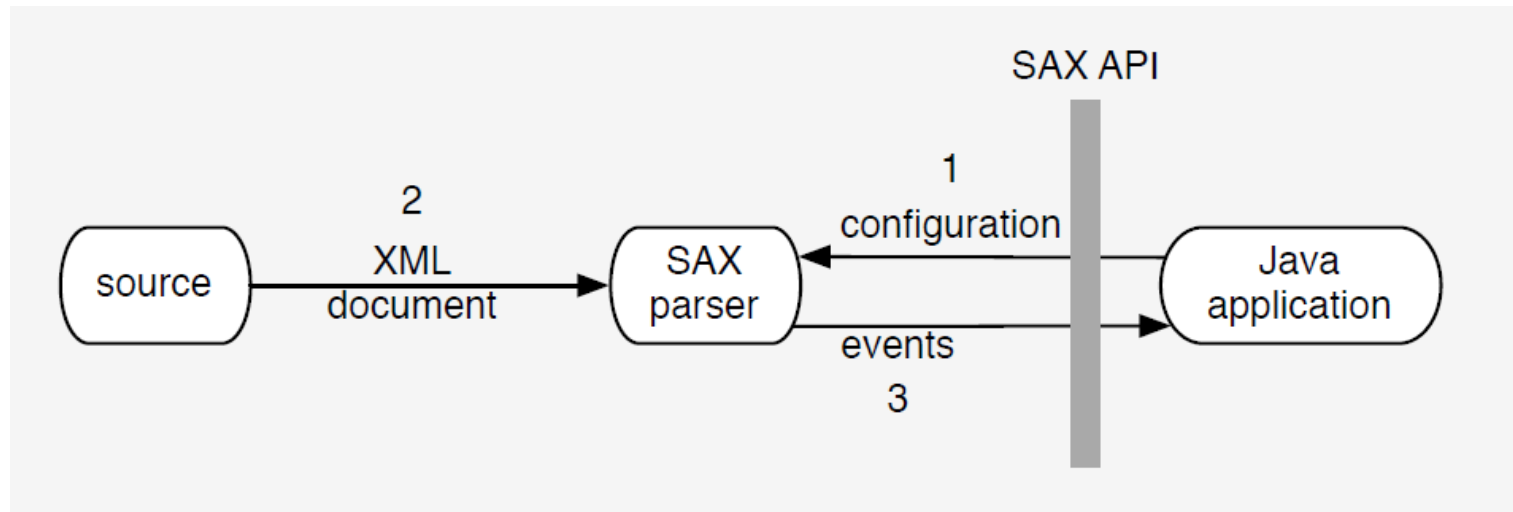
Agenda

- Rappel : validation
- SAX
 - Principe
 - Validation avec SAX
- DOM
 - Principe
 - Navigation et manipulation du DOM

Sax : présentation

- API basée sur des évènements de type “push” : le parser envoie les évènements vers l'application
- SAX n'est pas un standard à proprement parler, c'est plus un standard “de facto”.
- Première implementation en Java, mais a été adapté à d'autres langages
- Version 2.0.2 depuis le 27 avril 2004
- <http://www.saxproject.org/>

Flux de données



1. L'application instancie et configure le parser SAX
2. Le document XML est lu depuis la source
3. Des évènements sont envoyés à l'application (c-à-d. des méthodes callback sont appelées)

On répète les étapes 2 et 3.

Agenda

- Rappel : validation
- SAX
 - Principe
 - Validation avec SAX
- DOM
 - Principe
 - Navigation et manipulation du DOM

Sax : version la plus simple

```
import java.io.File; import java.io.IOException;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.parsers.SAXParser;
import javax.xml.parsers.SAXParserFactory;
import org.xml.sax.SAXException;

public class SimplestSaxParser {

    public static void main(String[] args) {
        try {
            SAXParserFactory factory = SAXParserFactory.newInstance();
            factory.setValidating(false);
            factory.setNamespaceAware(true);
            SAXParser sp = factory.newSAXParser();
            sp.getXMLReader().parse("howtoContent.xml" );
        }
        catch (ParserConfigurationException pce) {
            System.out.println("ParserConfigurationException : " + pce.getMessage()); }
        catch (IOException io) {System.out.println("IOException : " + io.getMessage()); }
        catch (SAXException se){System.out.println("SaxException : " + se.getMessage());}
    }
}
```

Documents XML

howto.xml : Valide et bien formé

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE howto SYSTEM "howto.dtd">
<howto>
  <topic>
    <title>Java</title>
    <url>http://www.rgagnon.com/topics/java-xml.html</url>
  </topic>
  <topic>
    <title>Power&lt;Builder</title>
    <url>http://www.rgagnon.com/topics/pb-powerscript.htm</url>
  </topic>
  <topic>
    <title>VBScript</title>
    <url>http://www.rgagnon.com/topics/wsh-vbs.html</url>
  </topic>
</howto>
```

howtoEndTag.xml Mal formé (maque un </topic>)

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE howto SYSTEM "howto.dtd">
<howto>
  <topic>
    <title>Java</title>
    <url>http://www.rgagnon.com/topics/java-xml.html</url>
  <topic>
    <title>Power&lt;Builder</title>
    <url>http://www.rgagnon.com/topics/pb-powerscript.htm</url>
  </topic>
  <topic>
    <title>VBScript</title>
    <url>http://www.rgagnon.com/topics/wsh-vbs.html</url>
  </topic>
</howto>
```

howtoContent.xml : Mal formé (< dans données)

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE howto SYSTEM "howto.dtd">
<howto>
  <topic>
    <title>Java < C++</title>
    <url>http://www.rgagnon.com/topics/java-xml.html</url>
  </topic>
  <topic>
    <title>Power&lt;Builder</title>
    <url>http://www.rgagnon.com/topics/pb-powerscript.htm</url>
  </topic>
  <topic>
    <title>VBScript</title>
    <url>http://www.rgagnon.com/topics/wsh-vbs.html</url>
  </topic>
</howto>
```

howtoTitle.xml : Bien formé mais invalide

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE howto SYSTEM "howto.dtd">
<howto>
  <topic>
    <title>Java</title>
    <title>C++</title>
    <url>http://www.rgagnon.com/topics/java-xml.html</url>
  </topic>
  <topic>
    <title>Power&lt;Builder</title>
    <url>http://www.rgagnon.com/topics/pb-powerscript.htm</url>
  </topic>
  <topic>
    <title>VBScript</title>
    <url>http://www.rgagnon.com/topics/wsh-vbs.html</url>
  </topic>
</howto>
```

Premier essai

- howto.xml : N'affiche rien
- howtoEndTag.xml :
 - [Fatal Error] howtoEndTag.xml:20:3: Le type d'élément "topic" doit se terminer par la balise de fin correspondante "</topic>".
 - SaxEXception : Le type d'élément "topic" doit se terminer par la balise de fin correspondante "</topic>".
- howtoContent.xml
 - [Fatal Error] howtoContent.xml:6:18: Le contenu des éléments doit inclure un balisage ou des caractères au format correct.
 - SaxEXception : Le contenu des éléments doit inclure un balisage ou des caractères au format correct.
- howtoTitle.xml : N'affiche rien

Premier essai (2)

- Les 3 premiers tests sont concluants et intuitifs
- Le 4^{ème} est plus perturbant.
 - Rien n'est affiché
 - En effet, nous avons désactivé la validation
 - Le parser va uniquement vérifier si le document est bien formé
 - C'est le cas ici, donc il ne dit rien

Fichier DTD associé

```
<!ELEMENT howto (topic*)>  
<!ELEMENT topic (title,url)>  
<!ELEMENT title (#PCDATA)>  
<!ELEMENT url (#PCDATA)>  
<!ELEMENT book (name)>
```

Activation de la validation

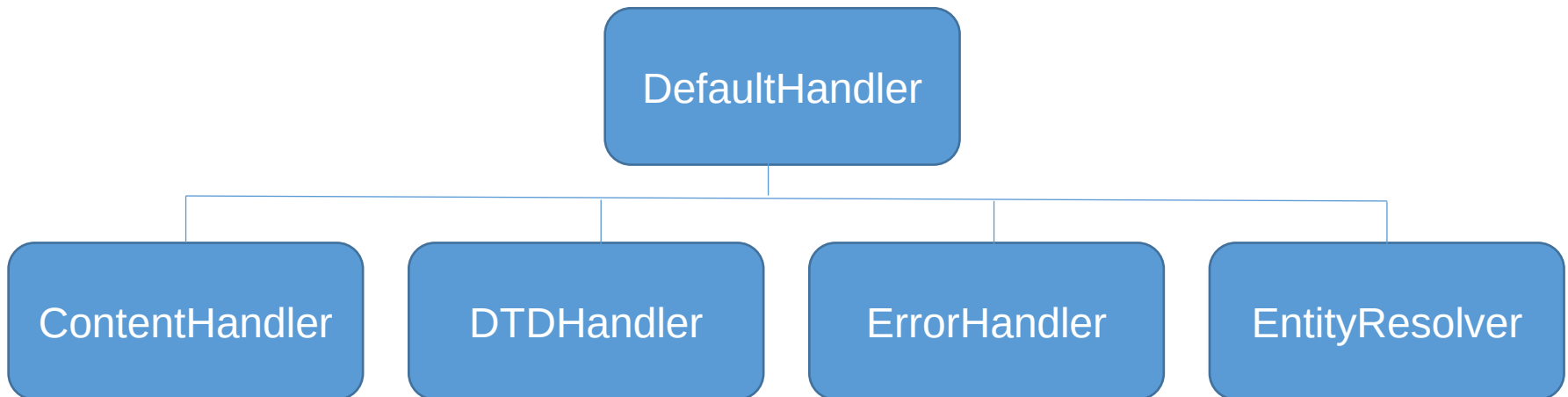
- Nous changeons la ligne **factory.setValidating(true);**
- `howto.xml` : N'affiche rien
- `howtoEndTag.xml` :
 - `SaxException` : Le type d'élément "topic" doit se terminer par la balise de fin correspondante "`</topic>`".
- `howtoContent.xml`
 - `SaxException` : Le contenu des éléments doit inclure un balisage ou des caractères au format correct.
- `howtoTitle.xml` :
 - Avertissement : la validation a été activée mais aucun élément `org.xml.sax.ErrorHandler` n'a été défini, ce qui devrait probablement être le cas. L'analyseur utilisera un gestionnaire d'erreurs par défaut pour imprimer les 0 premières erreurs. Appelez la méthode '`setErrorHandler`' pour résoudre ce problème.
 - Error: URI=file:///howtoTitle.xml Line=8: Le contenu du type d'élément "topic" doit correspondre à "(title,url)".

Explication

- Le fichier DTD est bien associé au XML
 - `<!DOCTYPE howto SYSTEM "howto.dtd">`
- La validation peut donc bien avoir lieu
- Le parser détecte que la règle topic (title,url) n'est pas respectée
 - Car il y a deux title
- Mais on ne lui a pas dit quoi faire avec.
- Il a besoin d'un *handler*

Les handlers

- Contiennent les méthodes callbacks qui sont appelées au fur et à mesure du parsing.
- Jusqu'ici, nous avons utilisé le *DefaultHandler*
 - Qui ne fait pas grand-chose
- Structure du DefaultHandler



Les handlers (2)

- ContentHandler
 - Méthodes liées au contenu
 - startElement (début de tag)
 - endElement (fin de tag)
 - characters (caractères ; contenu)
 - ignorableWhiteSpaces (espaces dus à l'indentation)
- DTDHandler
 - notationDecl (déclaration de notation)
 - unparsedEntityDecl (entités « unparsed » ; extérieures)

Les handlers (3)

- ErrorHandler
 - Warning (avertissement)
 - Error (erreur récupérable)
 - FatalError (erreur non récupérable)
- EntityResolver
 - resolveEntity (pour spécifier quoi faire avec l'entité)
- **Note** : dans le DefaultHandler, la plupart de ces méthodes sont vides, ou relancent simplement une exception

Spécifier un handler par défaut

```
SAXParser sp = factory.newSAXParser();
XMLReader reader = sp.getXMLReader();

reader.setErrorHandler(
    new ErrorHandler() {
        public void warning(SAXParseException e) throws SAXException {
            System.out.println("WARNING : " + e.getMessage()); // do nothing
        }

        public void error(SAXParseException e) throws SAXException {
            System.out.println("ERROR : " + e.getMessage());
            throw e;
        }

        public void fatalError(SAXParseException e) throws SAXException {
            System.out.println("FATAL : " + e.getMessage());
            throw e;
        }
    }
}
```

Deuxième essai

- `howto.xml` : N'affiche rien
- `howtoEndTag.xml` :
 - FATAL : Le type d'élément "topic" doit se terminer par la balise de fin correspondante "`</topic>`".
 - `SaxException` : Le type d'élément "topic" doit se terminer par la balise de fin correspondante "`</topic>`".
- `howtoContent.xml`
 - FATAL : Le contenu des éléments doit inclure un balisage ou des caractères au format correct.
 - `SaxException` : Le contenu des éléments doit inclure un balisage ou des caractères au format correct.
- `howtoTitle.xml` :
 - ERROR : Le contenu du type d'élément "topic" doit correspondre à "`(title,url)`".
 - `SaxException` : Le contenu du type d'élément "topic" doit correspondre à "`(title,url)`".

Autre méthode : classe externe

```
public class MyHandler extends DefaultHandler {  
  
    int cptTags = 0;  
  
    // Quelques méthodes du ContentHandler  
  
    public void characters(char[] ch, int start, int length) throws SAXException {  
        String chaine = new String(ch, start, length).trim();  
        if (chaine.length() > 0) {  
            System.out.println("@ Caractères : " + chaine);  
        }  
    }  
  
    public void startDocument() throws SAXException {  
        System.out.println("*** Début du document ***");  
    }  
  
    public void endDocument() throws SAXException {  
        System.out.println("*** Fin du document ***");  
    }  
}
```

Classe externe (suite)

```
public void startElement(java.lang.String uri, java.lang.String localName,  
    java.lang.String qName, Attributes attr) throws SAXException {  
    System.out.println("* Début d'un élément");  
    cptTags++;  
    System.out.println("++ compteur de tags : " + cptTags);  
    if (uri != null && uri.length() > 0) {  
        System.out.println(" uri : " + uri);  
    }  
    System.out.println(" nom complet : " + qName);  
    if (uri != null && uri.length() > 0) {  
        System.out.println(" nom complet : " + qName);  
    }  
    int nAttr = attr.getLength();  
    System.out.println(" nombre d'attributs : " + nAttr);  
    if (nAttr == 0) {  
        return;  
    }  
    for (int i = 0; i < nAttr; i++) {  
        System.out.println(" attribut n°" + i + " = " + attr.getLocalName(i)  
            + " avec valeur : " + attr.getValue(i));  
    }  
}
```

Classe externe (suite)

```
public void endElement(java.lang.String uri, java.lang.String localName,  
    java.lang.String qName) throws SAXException {  
    System.out.println("* Fin de l'élément " + localName);  
    cptTags++;  
    System.out.println("++ compteur de tags : " + cptTags);  
}
```

//Et d'autres du error handler

```
public void warning(SAXParseException e) throws SAXException {  
    System.out.println("WARNING : " + e.getMessage()); // do nothing  
}
```

```
public void error(SAXParseException e) throws SAXException {  
    System.out.println("ERROR : " + e.getMessage());  
    throw e;  
}
```

```
public void fatalError(SAXParseException e) throws SAXException {  
    System.out.println("FATAL : " + e.getMessage());  
    throw e;  
}
```


Et dans le main

```
SAXParser sp = factory.newSAXParser();  
sp.parse("howto.xml", new MyHandler());
```

Troisième essai

** Début du document **

* Début d'un élément

++ compteur de tags : 1

nom complet : howto

nombre d'attributs : 0

* Début d'un élément

++ compteur de tags : 2

nom complet : topic

nombre d'attributs : 0

* Début d'un élément

++ compteur de tags : 3

nom complet : title

nombre d'attributs : 0

@ Caractères : Java

* Fin de l'élément title

++ compteur de tags : 4

* Début d'un élément

++ compteur de tags : 5

nom complet : url

nombre d'attributs : 0

@ Caractères : <http://www.rgagnon.com/topics/java-xml.html>

...

Entités

- Les entités « classiques » sont gérés
 - < > ...
- Idem pour les entités numériques
- Par contre, si vous voulez utiliser une entité inconnue :
 - ERROR : L'entité "nbsp" était référencée, mais pas déclarée.
 - SaxEXception : L'entité "nbsp" était référencée, mais pas déclarée.
- Solution :
 - La déclarer dans le DTD
 - Si validation avec XSD, faire une référence vers un namespace dans lequel l'entité est déclarée.

Validation avec XSD

- Soit XSD interne

- Dans le XML :

```
<howto xsi:noNamespaceSchemaLocation="howto.xsd"  
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
```

- Dans le java :

```
SAXParser parser = factory.newSAXParser();  
parser.setProperty("http://java.sun.com/xml/jaxp/properties/schemaLanguage",  
"http://www.w3.org/2001/XMLSchema");
```

Validation avec XSD (2)

- Soit XSD externe

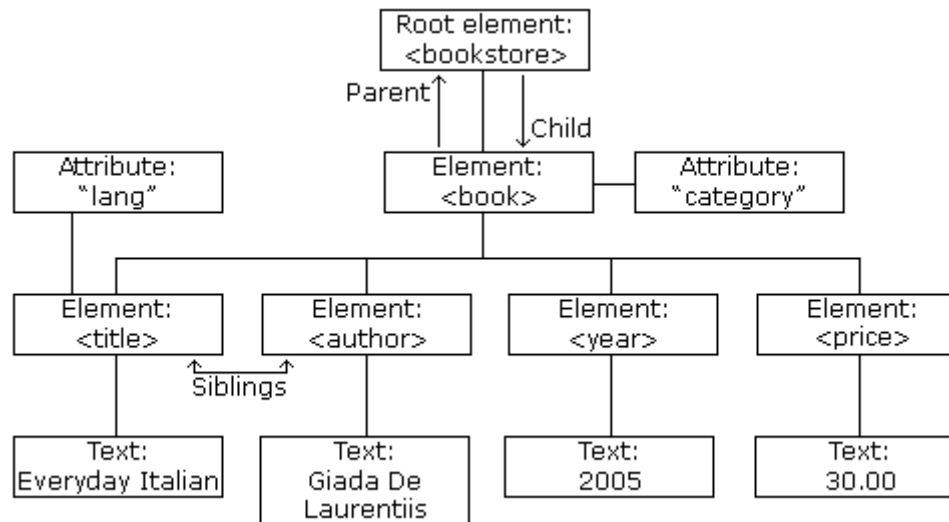
```
factory.setValidating(false);
SchemaFactory schemaFactory =
SchemaFactory.newInstance("http://www.w3.org/2001/XMLSchema");
SAXParser parser = null;
try {
    factory.setSchema(schemaFactory.newSchema(new Source[] {
        new StreamSource("howto.xsd"))));
    parser = factory.newSAXParser();
}
catch (SAXException se) {
    System.out.println("SCHEMA : " + se.getMessage());
    // problem in the XSD itself
    return false;
}
```

Agenda

- Rappel : validation
- SAX
 - Principe
 - Validation avec SAX
- DOM
 - Principe
 - Navigation et manipulation du DOM

DOM : Document object model

- Le principe est de construire un arbre à partir du XML



- Il est possible de construire un arbre :
 - Le document XML est bien formé
- L'arbre correspond au schéma (DTD ou XSD)
 - Le document XML est valide

DOM vs SAX

- Avantages de DOM
 - Plus rapide
 - La traversée de la structure est plus facile
 - Possibilité d'utiliser un langage de recherche (type XPath)
 - Possibilité de modifier la structure (et donc le XML sous-jacent)
- Inconvénients de DOM
 - Doit être chargé entièrement en mémoire (pas d'analyse « à la volée »)
 - Donc, plus gourmand en ressources
- On utilisera donc SAX si l'on souhaite valider un gros fichier (ou un fichier provenant du réseau)
- Et on utilisera DOM pour travailler la structure

Un simple parser DOM

```
public static void main(String[] args) {  
    DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();  
  
    try {  
        factory.setValidating(true);  
        DocumentBuilder builder = factory.newDocumentBuilder();  
  
        File fileXML = new File("howto.xml");  
  
        Document xml = builder.parse(fileXML);  
        Element root = xml.getDocumentElement();  
        System.out.println(root.getNodeName());  
    } catch (SAXParseException e){}  
  
    } catch (ParserConfigurationException e) { e.printStackTrace();  
    } catch (SAXException e) { e.printStackTrace();  
    } catch (IOException e) { e.printStackTrace(); }
```

Redéfinition des handlers

- Avec DOM, il est possible de redéfinir également les ErrorHandler et EntityResolver, de la même manière qu'avec SAX.
- Exemple :

```
DocumentBuilder builder = factory.newDocumentBuilder();
builder.setErrorHandler(
    new ErrorHandler() {
        public void warning(SAXParseException e) throws SAXException {
            System.out.println("WARNING : " + e.getMessage()); // do nothing
        }
        public void error(SAXParseException e) throws SAXException {
            System.out.println("ERROR : " + e.getMessage());
            throw e;
        }
        public void fatalError(SAXParseException e) throws SAXException {
            System.out.println("FATAL : " + e.getMessage());
            throw e;
        }
    }
);
```

Agenda

- Rappel : validation
- SAX
 - Principe
 - Validation avec SAX
- DOM
 - Principe
 - Navigation et manipulation du DOM

Manipulation du DOM

- Après le parsing, nous récupérons un objet *Document*
- Il est possible d'appeler des méthodes sur cet objet afin de parcourir l'arbre.
- Dans notre exemple, nous avons :

//Construction de l'arbre

```
Document xml = builder.parse(fileXML);
```

//Récupération de l'élément racine

```
Element root = xml.getDocumentElement();
```

//Affichera howto

```
System.out.println(root.getNodeName());
```

Navigation dans le DOM

- Autres méthodes utiles :
 - `getAttributes` : pour récupérer les attributs
 - Avec `getNodeName` et `getNodeValue` pour récupérer le nom et la valeur
 - `getChildNodes` : pour récupérer les nœuds enfants
 - `getElementsByTagName` : pour récupérer un ensemble de nœuds par leur nom, où qu'ils soient dans le document
 - `getFirstChild`, `getLastChild` : Pour récupérer, respectivement, le premier et le dernier nœud enfant
 - `getPreviousSibling`, `getNextSibling` : pour récupérer le frère de gauche ou de droite, respectivement
 - `getParentNode` : pour remonter dans la hiérarchie et récupérer le nœud parent

Modifier le DOM

- Il est également possible de modifier le DOM
 - `setNodeValue` : pour remplacer le contenu du nœud
 - `setAttribute` : pour modifier ou ajouter un attribut
 - `removeAttribute` : pour enlever un attribut
 - `appendChild` : ajouter un nœud enfant
 - `replaceChild` : remplacer un nœud enfant
 - `removeChild` : retirer un nœud enfant
- Pour en savoir plus, consultez la javadoc

Exemple : afficher le contenu du XML

```
public static String description(Node n, String tab){
    String str = new String();
    if (n instanceof Element) {
        Element element = (Element)n; //Casting
        str += "<" + n.getNodeName();

        if (n.getAttributes() != null && n.getAttributes().getLength() > 0){
            NamedNodeMap att = n.getAttributes();
            int nbAtt = att.getLength();
            for (int j = 0; j < nbAtt; j++){
                Node noeud = att.item(j);
                str += " " + noeud.getNodeName() + "=\"" + noeud.getNodeValue() + "\" ";
            }
        }
    }
}
```

Exemple : afficher le contenu du XML (2)

```
str += ">";
if (n.getChildNodes().getLength() == 1)
    str += n.getTextContent();
int nbChild = n.getChildNodes().getLength();
NodeList list = n.getChildNodes();
String tab2 = tab + "\t";
for(int i = 0; i < nbChild; i++){
    Node n2 = list.item(i);
    if (n2 instanceof Element){
        str += "\n " + tab2 + description(n2, tab2); //Recursion
    }
}

if(n.getChildNodes().getLength() < 2)
    str += "</" + n.getNodeName() + ">";
else
    str += "\n" + tab + "</" + n.getNodeName() + ">";
}
return str;
}
```


Attention aux espaces

- Imaginons le code suivant :

```
NodeList nodes = root.getChildNodes();
int nbNode = nodes.getLength();
for(int i = 0; i < nbNode; i++){
    Node n = nodes.item(i);
    System.out.println("* Enfant N°" + (i+1) + " : " + n.getNodeName() + " - " + n.getNodeValue());
}
```

- Le résultat est alors

```
* Enfant N°1 : #text -
```

```
* Enfant N°2 : topic - null
```

```
* Enfant N°3 : #text -
```

```
* Enfant N°4 : topic - null
```

```
* Enfant N°5 : #text -
```

```
* Enfant N°6 : topic - null
```

```
* Enfant N°7 : #text -
```

```
* Enfant N°8 : topic - null
```

```
* Enfant N°9 : #text -
```

De retour dans le XML

```
<howto>
  <topic>
    <title>Java</title>
    <url>http://www.rgagnon.com/topics/java-xml.html</url>
  </topic>
  <topic>
    <title>PowerBuilder</title>
    <url>http://www.rgagnon.com/topics/pb-powerscript.htm</url>
  </topic>
  ...
```

- Le problème : L'espace (et le retour chariot) entre les balises `<howto>` et `<topic>` ainsi qu'entre `</topic>` et `<topic>` est du texte, donc un nœud !

Solution

- Il suffit simplement de déclarer que certains caractères ne doivent pas être pris en compte dans la structure
- SAX avait la méthode `ignoreWhitespace`, appelée justement dans ce cas
- En DOM, il suffit simplement d'appeler :
`factory.setIgnoringElementContentWhitespace(true);`

Réécriture après modification

- Puisqu'il est possible de modifier le DOM d'un fichier XML, il peut être intéressant de réécrire le document XML correspondant au DOM modifié.
- Cela peut se faire, par exemple, à l'aide du code suivant :

```
TransformerFactory transformerFactory = TransformerFactory.newInstance();
Transformer transformer = transformerFactory.newTransformer();
DOMSource source = new DOMSource(domTree);
StreamResult result = new StreamResult(new File(filepath));
transformer.transform(source, result);
```

DOM et autres langages

- Nous avons vu DOM via son API Java
- Il peut aussi être implémenté via d'autres langages
 - En particulier JavaScript
- La syntaxe est un peu différente, mais le concept reste identique
 - Attention toutefois à la gestion des espaces
 - IE9 et précédentes retirent les espaces ignorables
 - IE10 et suivantes les laissent